

SISTEMAS DE INFORMACIÓN II

BIENVENIDOS!!



Docente:

Sonia Ana Ridjan

UADE

AGENDA

UADE

- ☐ Proceso – Procedimiento
- ☐ Ingeniería de Software
- ☐ Ingeniería de Requisitos
- ☐ Requisitos
- ☐ Roles en Proyectos de Software
- ☐ Dominio del problema
- ☐ Ejercicios en clase
- ☐ Bibliografía
- ☐ Dudas y preguntas

¿QUÉ ES UN PROCESO?

ENTRADA



TRANSFORMACIÓN



SALIDA



- ✓ Seleccionar las tablas de madera necesarias.
- ✓ Medir y marcar las piezas según el plano de fabricación.
- ✓ Cortar las patas, el asiento y el respaldo.
- ✓ Lijar todas las piezas para eliminar imperfecciones.
- ✓ Ensamblar las patas con el asiento utilizando cola y tornillos.
- ✓ Fijar el respaldo a la estructura.
- ✓ Verificar que la silla quede firme y nivelada.
- ✓ Aplicar barniz o pintura.
- ✓ Dejar secar el acabado.
- ✓ Realizar una inspección final y embalar la silla.



¿QUÉ ES UN PROCESO?

Conjunto de actividades que **se relacionan entre sí**, que **trabajan para el logro de un objetivo en común** y, que a su vez **poseen entradas** que **sufren una transformación** para **generar salidas**.

Características de un proceso

- ✓ Tiene **un objetivo**
- ✓ Tiene **entradas**
- ✓ Tiene **salidas**
- ✓ Está formado por **varias actividades /etapas**
- ✓ Puede involucrar **personas, herramientas y recursos**
- ✓ Un proceso es **dinámico**, evoluciona y puede cambiar con el tiempo.



EJEMPLO DE PROCESO

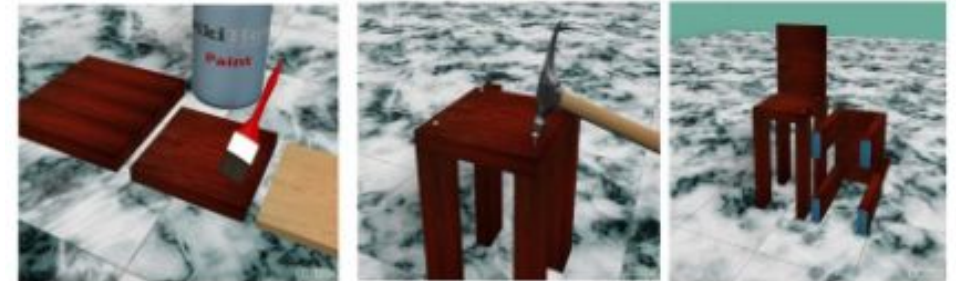
Empresa que se dedica a la fabricación de muebles artesanales en madera:

- ✓ **Proceso análisis del pedido del cliente:** se analiza lo que hay que fabricar y fecha de entrega.
- ✓ **Proceso de compras:** se solicita a los proveedores todos los elementos que se necesitan para la construcción de los muebles.
- ✓ **Proceso de producción:** se procede a la construcción de los muebles solicitado siguiendo los procedimientos que corresponda.
- ✓ **Proceso entrega del producto:** coordinar día y horario y transporte de los muebles.
- ✓ **Proceso postventa:** en la que se atenderán a las posibles dudas, sugerencias, quejas o reclamos que puedan existir una vez entregado los muebles contruidos.

PROCEDIMIENTO

Ejemplo:

- ✓ Para la base de la silla modelo M20 cortar trozo de madera cuadrada de 40x45 cm
- ✓ Para las patas de las sillas, cortar 4 trozos de madera de largo 50cm.
- ✓ Para el respaldo de la silla cortar un trozo de madera de 60x40 cm
- ✓ Lijar cada pieza.
- ✓ Barnizar cada pieza
- ✓ Ensamblar cada pieza con los tornillos.
- ✓ Etc.



¿QUÉ ES UN PROCEDIMIENTO?

Conjunto de pasos que se deben seguir para realizar una actividad o parte de un proceso.

Características:

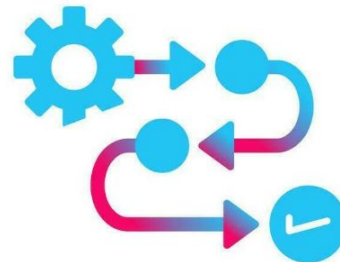
- ✓ Define **pasos concretos**
- ✓ Los pasos deben tener un **orden determinado**
- ✓ Es **más detallado**
- ✓ Es **repetible**
- ✓ Busca que la tarea se realice **siempre de la misma manera**
- ✓ Los procedimientos establecen **cómo se hace algo**
- ✓ **Forman parte de un proceso**



PROCESO Y PROCEDIMIENTO

El **proceso** responde a la pregunta **¿Qué etapas tiene el trabajo?**

El **procedimiento** responde a la pregunta **¿Cómo se realiza cada una de esas etapas?**



Qué es Software?

El **IEEE** define software como:

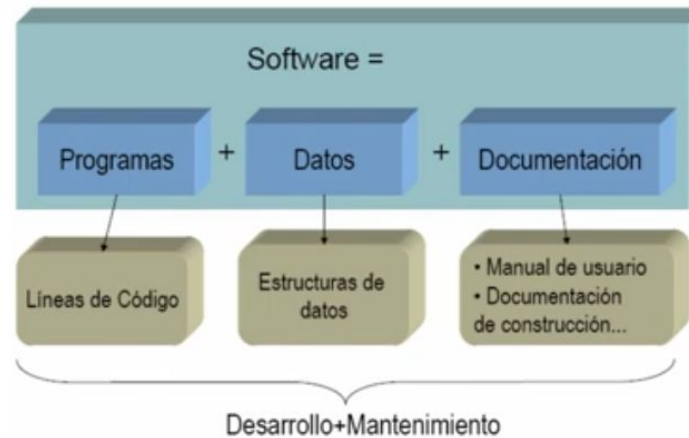
Programas de computadora, **procedimientos**, **reglas** y **documentación** asociada, así como **los datos** relacionados con la operación de un sistema informático.

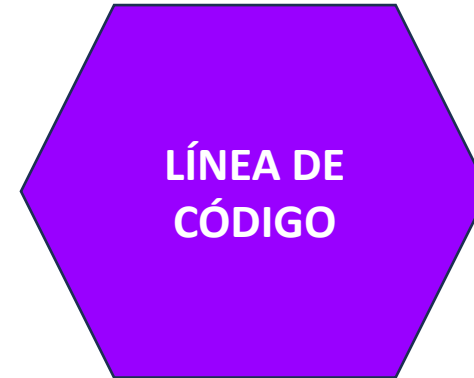
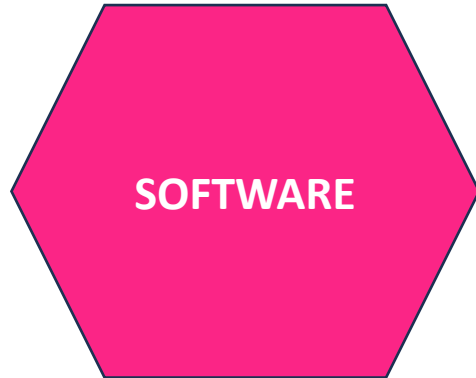


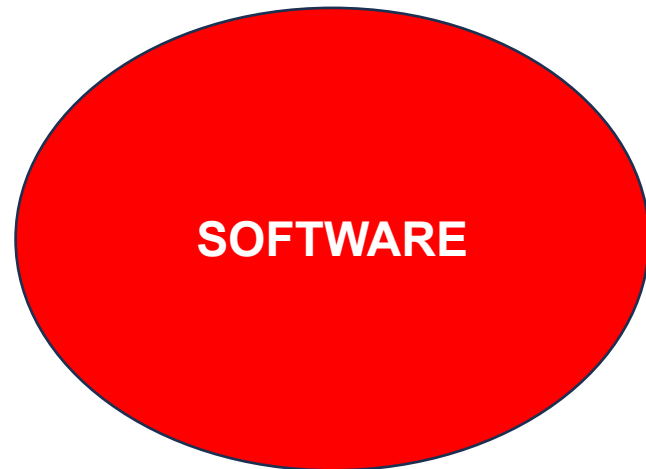
Pressman define el Software como:

El **conjunto de instrucciones** (programas de computadora), estructuras de **datos y documentación asociada** que describen cómo deben utilizarse los programas para que una computadora realice determinadas tareas.

- 1) **Programas:** conjunto de instrucciones que ejecuta la computadora para realizar una tarea.
- 2) **Estructuras de datos:** forma en que se organiza y almacenan los datos y la información para que los programas puedan utilizarla. (bases de datos , archivos, etc)
- 3) **Documentación:** información descriptiva tanto en papel como en formas virtuales que describen la operación y uso de los programas.







INGENIERÍA DE SOFTWARE

El concepto **"Ingeniería de Software"** se propuso por primera vez en 1968 por Fritz Bauer, en la primera conferencia de software organizada por la OTAN, este término se utilizó para describir los conocimientos que debían desarrollarse para dar solución a la **"Crisis del Software"**.



- ✓ Proyectos que se retrasaban mucho o se cancelaban
- ✓ Costos mucho mayores a los planificados
- ✓ Programas con gran cantidad de errores
- ✓ Software difícil de entender o modificar
- ✓ Sistemas que no cumplían con lo que necesitaba usuario

Objetivo

Aplicar prácticas similares de otras ingenierías al desarrollo de software.

- Planificación del proyecto
- Definición de procesos
- Documentación
- Control de calidad
- Uso de métodos y herramientas



INGENIERÍA DE SOFTWARE

UADE

"La **Ingeniería del Software** es el establecimiento y uso de principios de ingeniería sólidamente fundados con el fin de **obtener software de manera económica, que sea confiable y que funcione eficientemente en máquinas reales.**"(Fritz Bauer, 1968)

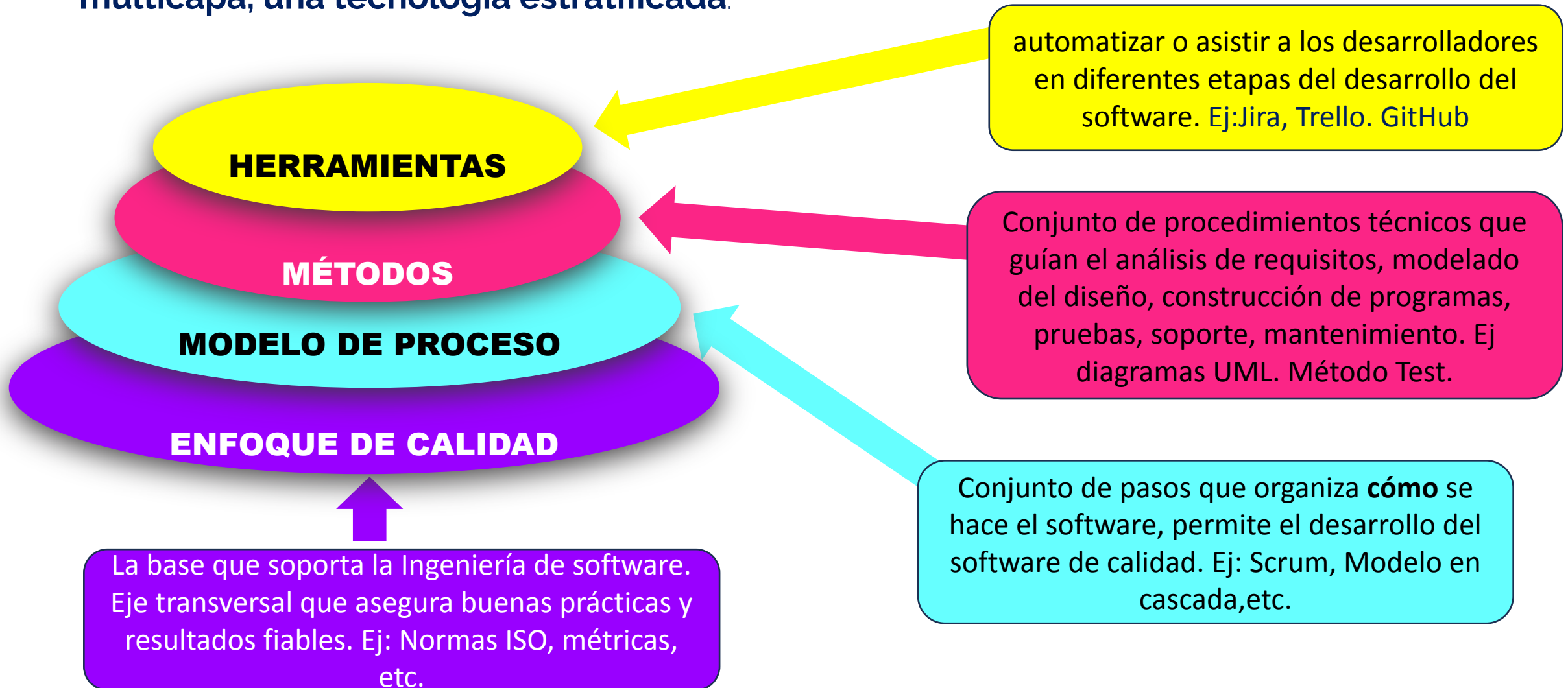
La ingeniería de software se define como la aplicación de un enfoque sistemático, disciplinado y cuantificable al desarrollo, operación y mantenimiento de software; es decir, la aplicación de la ingeniería al software (**Pressman, 2010**)

La ingeniería de software, es una disciplina de ingeniería que se interesa por todos los aspectos de la producción de software, desde las primeras etapas de la especificación del sistema hasta el mantenimiento del sistema después de que se pone en operación (**Sommerville, 2011**) .



INGENIERÍA DE SOFTWARE

La ingeniería de software es una tecnología multicapa, una tecnología estratificada:



Proceso de Software:

Guía estructurada para desarrollar software.

Conjunto de pasos ordenados y jerárquicos: de lo más general (actividad) a lo más concreto (tarea), con el fin de asegurar que el producto final sea de calidad, útil y mantenible.

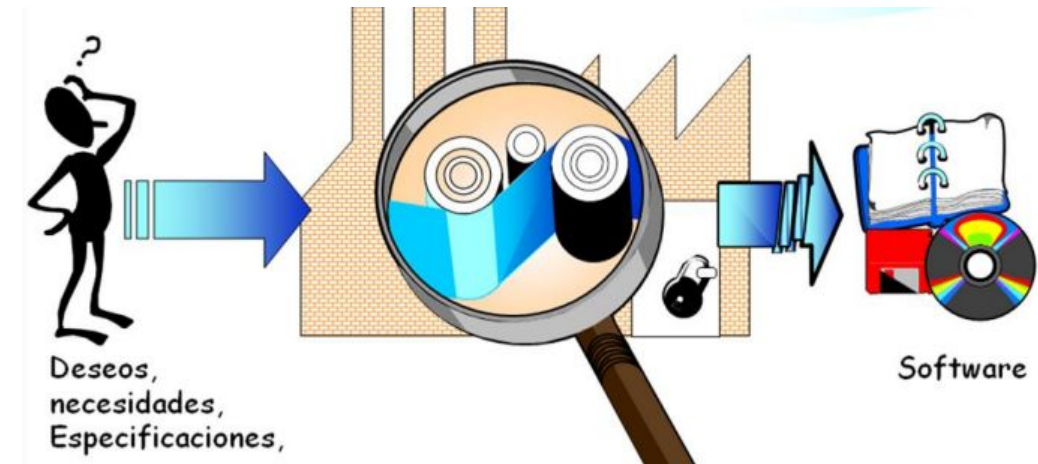
Nivel	Definición	Ejemplos
Actividad	Son las Fases generales que conforman el proceso de software	Comunicación con el cliente Planificación del proyecto Modelado del software Construcción (codificación y prueba) Despliegue del producto
Acción	Conjunto de pasos relacionados dentro de una actividad.	Identificar requerimientos Diseñar la interfaz de usuario Definir la arquitectura del sistema Escribir los casos de prueba Preparar manuales de usuario
Tarea	Paso muy específico que produce un resultado concreto . Se puede medir y asignar fácilmente.	Redactar un formulario de login Realizar una prueba unitaria de la función "Alta de cliente en la BD" Diseñar el botón de "Enviar" Revisar el código con otra persona (code review) Subir el código al repositorio en GitHub

Al **Proceso de Desarrollo de Software** también se le conoce como **Ciclo de vida del Software**.

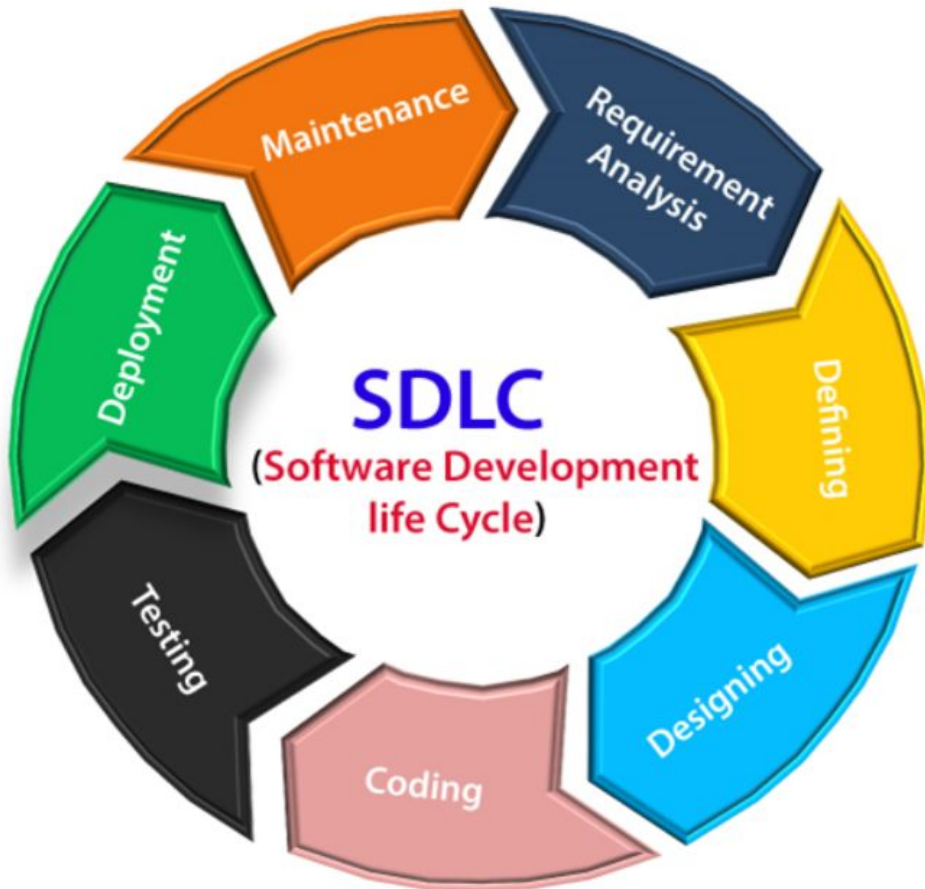
Ciclo de vida del Software:

(**SDLC** – Systems Development Life Cycle)

- ❑ Es una secuencia de etapas estructuradas bien definidas que contiene los procesos, actividades y tareas relacionadas con el desarrollo y mantenimiento de un producto de software, abarcando la vida completa del software, desde la definición de los requisitos hasta la finalización de su uso.



Ciclo de vida del software:





Definición de necesidades:

- El cliente solicita un producto de software determinado.
- Surge una necesidad de mejorar una tarea, solucionar un problema, mejorar un software ya existente, reemplazar un software existente, etc.



“Qué problema se necesita resolver o mejorar “



Planificación y análisis de los requisitos del software:

“Qué debe hacer el producto de software”

- La recolección de todos los requisitos.
- Se analizan los todos requisitos que piden los clientes .
- Depuración (cuales están poco claros, incompletos, ambiguos o contradictorios). Se indaga en profundidad y se hacen demostraciones prácticas incluyendo a los usuarios clave.
- Clasificación de requisitos en requisitos del usuario, requisitos funcionales y requisitos del sistema.



Planificación y análisis de los requisitos del software:

En esta fase el equipo del proyecto asigna recursos y planifica el tiempo de duración del proyecto.

Se buscan limitaciones del producto y se identifican los impactos del proyecto sobre toda la organización en su conjunto.

Estudio de factibilidad

Se analiza que parte del software cubre los requisitos de cada usuario. Se investiga la viabilidad financiera y tecnológica.





Diseño:

“Cómo va a funcionar el producto de software”

El diseño del software es realmente un proceso de varios pasos que se centra en los siguientes puntos principales:

- ✓ Estructura de datos.
- ✓ Diseño funcional (Modelado)
- ✓ Arquitectura de software.
- ✓ Representaciones de interfaz.
- ✓ Detalle procedimental (algoritmo).

Se hace un diseño lógico y físico.



El proceso del diseño traduce requisitos en una representación del software donde se pueda evaluar su calidad antes de que comience la codificación.



Codificación:

Es la “fase de programación” o “fase de desarrollo” se desarrollan programas ejecutables

Al final de esta fase se puede obtener un PMV (Producto mínimo viable) o el software completamente desarrollado y listo para implementarse.

Integración: el software puede necesitar estar integrado con bibliotecas, bases de datos o con otros programas. Esta fase del SDLC integra el software con las entidades del mundo exterior.





Pruebas:

La fase de pruebas se centra en los procesos lógicos internos del software, asegurando que todas las sentencias se han comprobado, y en los procesos externos funcionales; es decir, realizar las pruebas para la detección de errores y asegurar que la entrada definida produce resultados reales de acuerdo con los resultados requeridos.





Despliegue:

Es la fase de puesta en producción del software desarrollado.

Se instala el software, se evalúa la integración, la adaptabilidad, la portabilidad y se instalan las configuraciones posteriores necesarias.

El despliegue de software, es un proceso en sí mismo, implica una serie de pasos necesarios para entregar e instalar aplicaciones de forma correcta y eficiente.





Despliegue:

Planificación de la implementación: ¿Cómo y cuándo se va a desplegar? ¿Qué dependencias hay? ¿Quién es responsable?

Preparación del entorno del cliente: Hardware, sistema operativo, redes, permisos, software de base.

Instalación del software y configuración: Puede implicar integración con otros sistemas existentes.

Pruebas de aceptación del usuario (UAT): Validación del sistema en condiciones reales. Usuarios verifican si el sistema cumple los requisitos.

Transferencia de conocimiento: Documentación entregada, manuales, capacitación.

Mantenimiento inicial y monitoreo post-despliegue: Se da soporte técnico y se recopilan incidentes. Se definen responsables de mantenimiento y mejora.

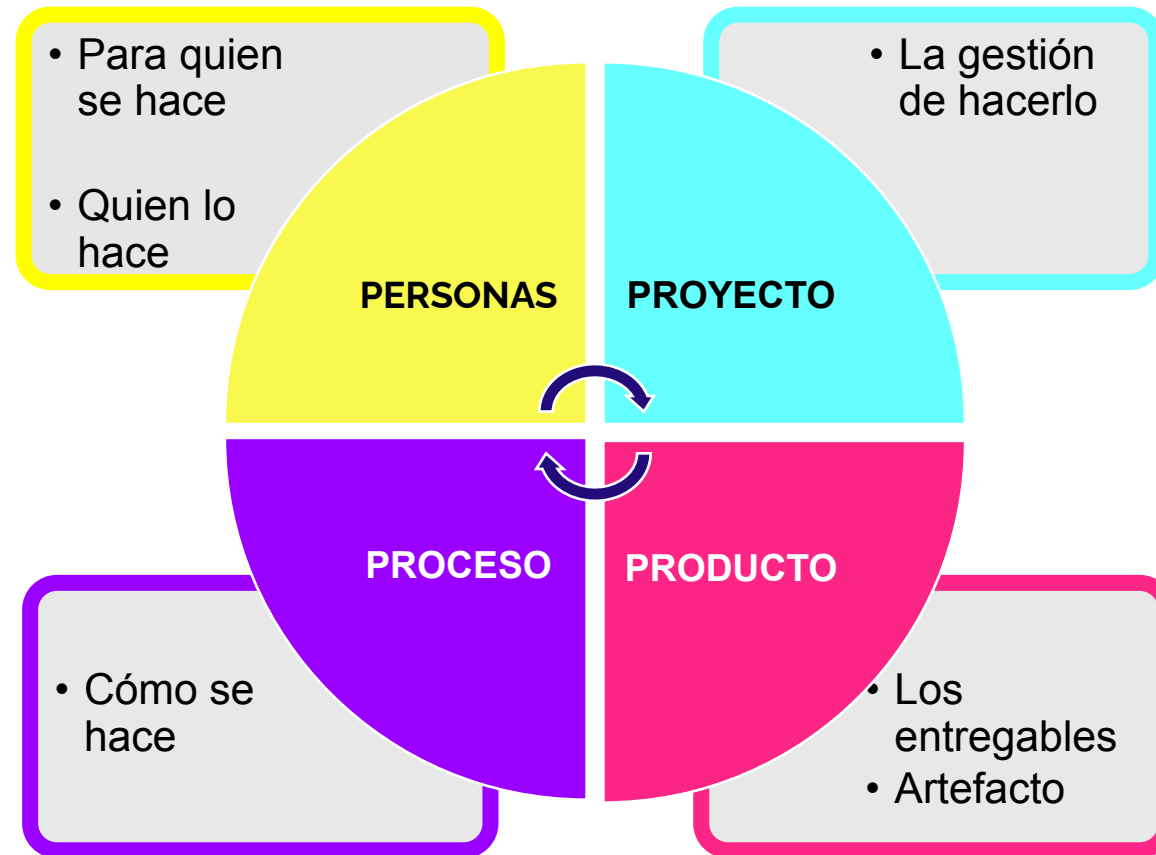


Mantenimiento/Evolución:

- El software sufrirá cambios después de ser entregado al cliente.
- Se corrigen errores que aparecen en el uso real.
- Se aplican mejoras, actualizaciones y adaptaciones.



- ❑ La ingeniería de software no es solo programar: **es planificar, organizar, liderar y controlar un proceso que transforme una necesidad en un producto funcional**. Para lograrlo con éxito, existen **cuatro factores clave que intervienen en cualquier proyecto de software**. Estos factores son conocidos como **las 4 P**:



Personas :

"El software lo desarrollan personas. El proceso y la tecnología solo ayudan."

Roger Pressman

- Las personas son el recurso más importante de cualquier proyecto
- Incluye desarrolladores, analistas, testers, líderes de proyecto, usuarios, etc.
- Es clave que estén bien organizadas, motivadas y capacitadas.
- Modelos como el equipo de construcción de software de Cockburn o los roles de Scrum (Product Owner, Scrum Master, Equipo de Desarrollo) ayudan a estructurar correctamente los equipos.

Buenas prácticas:

- ✓ Formación continua
- ✓ Comunicación efectiva
- ✓ Equipos pequeños y multidisciplinarios



Producto

"Para construir algo correctamente, primero hay que entender qué se va a construir."

Ian Sommerville.

- Se refiere al **software que se va a desarrollar** y a las **necesidades que debe cubrir**.
- Incluye los requisitos funcionales (qué hace el software) y no funcionales (calidad, rendimiento, usabilidad).
- Es fundamental contar con una **buena especificación de requisitos**, que puede evolucionar con el tiempo.

Buenas prácticas:

- ✓ Reuniones con el cliente o usuario
- ✓ Prototipos o mockups
- ✓ Historias de usuario y casos de uso



Proyecto

"Administrar un proyecto de software es equilibrar tiempo, costo, alcance y calidad."

Roger Pressman

- Se refiere a la **gestión del proyecto**: cómo se planifica, organiza, controla y ajusta el trabajo para lograr los objetivos.
- **Herramientas clave:**
 - Diagramas de Gantt
 - Planificación de tareas
 - Estimación de esfuerzo (p. ej., Puntos de historia, COCOMO)
 - Control de riesgos
 - Control de versiones (Git, por ejemplo)

Buenas prácticas:

- ✓ Dividir el proyecto en etapas
- ✓ Medir el avance (porcentaje completado, velocidad del equipo)
- ✓ Gestionar el cambio y los riesgos



Proceso

"Un buen proceso no garantiza el éxito, pero uno malo sí garantiza el fracaso."

Watts Humphrey

Es el **conjunto de actividades y tareas** organizadas que se siguen para desarrollar el software.

- Incluye metodologías como:
 - Cascada (Waterfall)
 - Prototipado
 - Espiral
 - Ágil (Scrum, XP, Kanban)
- Cada proceso tiene sus fortalezas y se elige según el contexto del proyecto (tiempo, presupuesto, complejidad, grado de cambio en los requisitos, etc.).

Buenas prácticas:

- ✓ Seleccionar el modelo adecuado
- ✓ Definir roles y etapas claras
- ✓ Revisar y mejorar el proceso durante el desarrollo



- ❑ Un proyecto de software involucra a distintos perfiles, con responsabilidades específicas. Estos roles pueden variar un poco según la metodología (ágil o tradicional), pero en líneas generales encontramos:

Rol	Función general
Cliente / Sponsor	Financia o encarga el sistema
Usuario final	Utiliza el sistema en su trabajo diario
Analista de Requisitos / Funcional	Recoge, analiza y documenta los requisitos
Desarrollador / Programador	Construye el sistema según los requisitos
Tester / QA (Calidad)	Verifica que el sistema cumpla con los requisitos
Gerente de Proyecto / PM	Coordina tareas, tiempos, recursos
Diseñador UX/UI	Diseña la interfaz y experiencia del usuario
Arquitecto de Software	Define estructura técnica y decisiones clave
Product Owner (en Ágil)	Representa al cliente y prioriza funcionalidades
Scrum Master (en Ágil)	Facilita el proceso y elimina obstáculos

ROLES EN LA DEFINICIÓN DE REQUISITOS DE SOFTWARE

- ❑ Roles claves que participan directamente en la definición de requisitos :

Rol	Funciones específicas
Cliente	Define necesidades del negocio y objetivos generales
Usuario final	Explica procesos reales, aporta necesidades funcionales. Persona que utiliza el sistema.
Analista de Requisitos	Traductor entre negocio y tecnología; recoge, valida y documenta
Product Owner	Prioriza requisitos en función del valor de negocio (Ágil)
Gerente de Proyecto	Supervisa que los requisitos estén alineados al alcance y tiempos
Arquitecto de Software	Evalúa la viabilidad técnica de los requisitos
Tester / QA	Participa para definir criterios de aceptación desde el inicio
Reguladores / Legales	Aportan requisitos normativos o legales obligatorios

STAKEHOLDERS

- ❑ Un **stakeholder** (parte interesada) es **cualquier persona, grupo u organización que afecta o es afectada por el sistema que se está desarrollando.**
- ❑ Estos son los que **entienden cómo funciona el negocio** para el cual yo realizo el desarrollo. **Entiende la lógica del negocio y las necesidades** que se requieren cubrir con el sistema.
- ❑ Ellos **también conocen las restricciones de tiempo y dinero** que existen en el proyecto.



CLASIFICACIÓN DE STAKEHOLDERS

Tipo de stakeholder	Rol principal	Ejemplo práctico
Cliente	Financia o encarga el sistema	Empresa que contrata el desarrollo
Usuario final	Usa el sistema	Empleado que opera una app
Regulador	Impone normativas	Ente estatal o auditoría externa
Tester	Valida el sistema	QA, analista funcional
Administrador	Mantiene el sistema	Soporte técnico
Sponsor/ Patrocinador	Apoya políticamente el proyecto	Gerente de área

CLASIFICACIÓN DE STAKEHOLDERS

1. Stakeholders internos

Personas dentro de la organización que desarrolla o usa el sistema.

Ejemplos:

- ✓ Analistas
- ✓ Programadores
- ✓ Testers
- ✓ Gerentes de proyecto
- ✓ Empleados de la empresa

2. Stakeholders externos

Personas u organizaciones fuera de la empresa que influyen el sistema.

Ejemplos:

- ✓ Clientes
- ✓ Usuarios finales
- ✓ Proveedores
- ✓ Entidades regulatorias

¿Son lo mismo stakeholders, usuarios y clientes?

- ☐ No son lo mismo.
- ☐ Sí son conceptos relacionados, pero no equivalentes.

Stakeholder (parte interesada)

Definición:

Cualquier persona, grupo u organización que tiene interés, poder o se ve afectado por el sistema, directa o indirectamente.

Ejemplos:

Un tester: no usa el sistema final, pero lo valida.

Un regulador del Estado: impone normas que el sistema debe cumplir.

Un proveedor externo: necesita integrarse con el sistema.

Un usuario o cliente: también son stakeholders, pero no los únicos.

2. Usuario del sistema

Definición:

Persona que interactúa directamente con el sistema para realizar tareas específicas.

Ejemplos:

Recepcionista que da turnos en una clínica desde una plataforma.

Empleado bancario que carga datos en el sistema de cuentas.

Pasajero que reserva un boleto desde una app.

Importante: un sistema puede tener varios tipos de usuarios, con distintos permisos o funciones.

3. Cliente

Definición:

Persona o entidad que financia, contrata o aprueba el desarrollo del sistema.

Ejemplos:

Dueño de un gimnasio que encarga una app para gestionar socios.

Empresa de transporte que compra un sistema de gestión de flota.

Gobierno que contrata un software para gestionar becas.

A veces, el cliente no usa el sistema, pero tiene poder de decisión.

¿Son lo mismo stakeholders, usuarios y clientes?

- ☐ No son lo mismo.
- ☐ Sí son conceptos relacionados, pero no equivalentes.

Concepto	¿Qué es?	¿Participa en el sistema?	¿Siempre usa el sistema?
Stakeholder	Toda persona u organización que afecta o es afectada por el sistema	Sí	No necesariamente
Usuario del sistema	Persona que usa el sistema directamente	Sí	Sí
Cliente	Persona o entidad que encarga, paga o financia el sistema	Sí	No necesariamente

EJEMPLO

Caso: Se desarrolla un sistema de gestión de turnos médicos en un Sanatorio.

Tipo de Stakeholder	Rol	¿Es usuario?	¿Es cliente?
Gerente del centro médico	Cliente	No	Sí
Paciente que reserva turnos	Usuario final	Sí	No
Recepcionista que gestiona los turnos	Usuario interno	Sí	No
Programador del sistema	Desarrollador	No	No
Ministerio de Salud	Regulador	No	No
Dueños del Sanatorio	Cliente	No	Sí

❑ ¿Qué es un requisito?

Un requisito **es una declaración que traduce una necesidad, restricción o expectativa de los clientes** sobre un sistema .

Reflejan las necesidades de los clientes que **ayude a resolver algún problema a través de un Sistema de Información.**

Definen las funciones que el sistema será capaz de realizar.

Los requisitos **no son estáticos**.

Cambian por nuevas necesidades, tecnología, leyes, errores, etc.



❑ REQUISITO es sinónimo de REQUERIMIENTOS?

En ingeniería de software y en español técnico :

NO son lo mismo,

aunque en la práctica muchas veces se usan como sinónimos.

Término	Definición	En qué contexto se usa
Requerimiento (needs)	Es la necesidad o expectativa que expresa un usuario, cliente o parte interesada. Puede estar en lenguaje natural, incompleto o ambiguo.	Es la etapa de recolectar información del negocio y o usuarios(elicitación)
Requisito (requirement)	Es un enunciado claro, verificable y documentado que especifica qué debe hacer el sistema o cómo debe comportarse. Es la respuesta técnica y concreta a esa necesidad.	Etapa de análisis, especificación y validación.

Autores que respaldan esta distinción:

Sommerville (Ingeniería de Software):

“El término **requerimiento** suele usarse de forma informal, mientras que **requisito** se refiere al resultado de un proceso formalizado de documentación.”

Pohl y Rupp (Fundamentos de Ingeniería de Requisitos):

“Los **requerimientos** expresan deseos, mientras que los **requisitos** son el resultado de su análisis y estructuración.”

❑ Relación entre REQUISITO y REQUERIMIENTOS

Un **requerimiento** del usuario se **transforma** en uno o varios **requisitos del sistema** mediante el trabajo del analista /ingenieros de requisitos.

Ejemplo:

Requerimiento del usuario:

“Quiero que el sistema sea fácil de usar.”

Requisito derivado (especificación técnica):

“El sistema deberá permitir completar una orden en no más de 3 pasos y con un máximo de 2 clics por pantalla.”

INGENIERÍA DE REQUISITOS

Origen

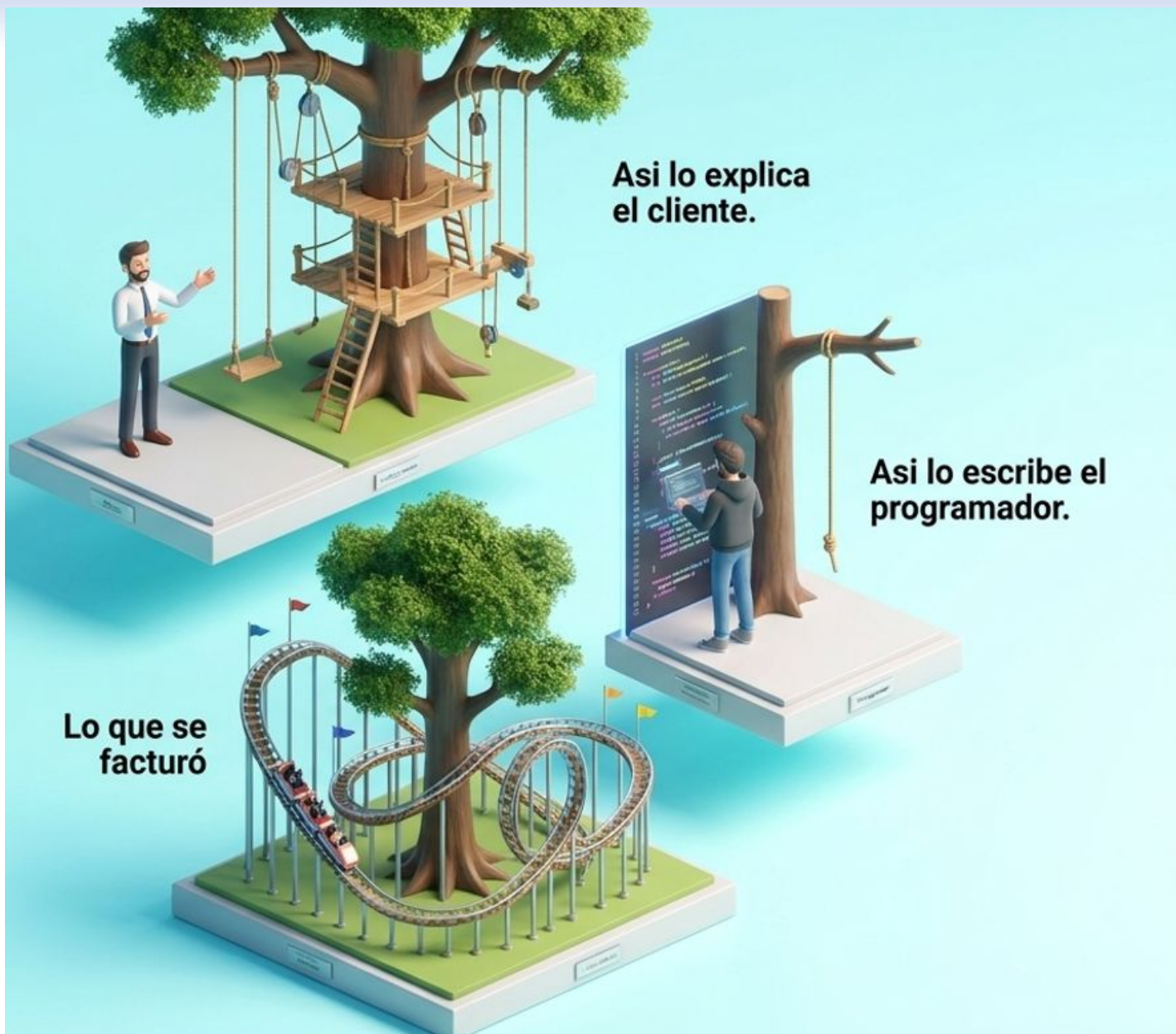
❑ Década del 80

Surgen Investigaciones que analizan cómo identificar y documentar requisitos de software de forma sistemática.

❑ Década del 90

Se consolida la Ingeniería de Requisitos como un área específica dentro de la Ingeniería de Software. (IEEE - Institute of Electrical and Electronics Engineers)





La Paradoja del Software

"El problema más grave en un proyecto de software es construir el sistema correcto de manera incorrecta... o construir correctamente el sistema equivocado."

Ejemplo

1. Lo que **necesita el cliente**: "Necesito que los clientes puedan reservar turnos médicos online."
2. Lo que **entiende el analista**: El analista interpreta: "El sistema debe permitir registrar pacientes y cargar turnos manualmente."
3. el **programador** desarrolla lo que le indica el analista.
4. Resultado

La clínica recibe el sistema y dice:

✗ "Esto funciona, pero no es lo que necesitábamos. Queríamos que los pacientes pudieran reservar sus propios turnos desde Internet."

- ❑ **Es la disciplina encargada de identificar las necesidades de los Usuarios, transformarlos en especificaciones precisas, no ambiguas, consistentes y completas del comportamiento del sistema, incluyendo funciones, interfaces, rendimiento y limitaciones. Documentar, validar y gestionar los requisitos a lo largo del ciclo de vida del proyecto de software.**
- ❑ Sirve como base para acuerdos comunes entre todas las partes involucradas y en donde se describen las funciones que realizará el sistema.
- ❑ Utiliza una combinación de métodos, herramientas y actores, cuyo producto es un modelo del cual se genera un documento de requerimientos (especificaciones)."

Definición:

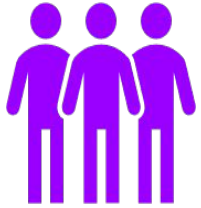
El proceso de descubrir, analizar, documentar y verificar los servicios que un sistema debe proporcionar y las restricciones bajo las cuales debe operar.

Sommerville, *Software Engineering*.

El conjunto de tareas que permiten comprender qué necesita el cliente y especificar esas necesidades de manera clara y verificable.

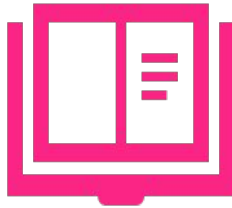
Pressman, *Software Engineering: A Practitioner's Approach*.

FUENTES DE REQUISITOS



STAKEHOLDERS

- Persona,
- Grupo u Organizaciones que afecta o es afectada directa o indirectamente por el producto de software que se está desarrollando.



DOCUMENTOS

- Documentación existente.
- Manuales, reglamentos, formularios, reportes.
- Permiten entender cómo se trabaja actualmente y qué requisitos son obligatorios.



SISTEMAS ACTUALES

- Analizar aplicaciones o procesos ya en uso.
- Identificar qué se debe mantener, mejorar o eliminar.



OBSERVACIÓN DIRECTA

- Ver cómo los usuarios realizan su trabajo.
- Detectar necesidades que ellos mismos no mencionan (requisitos tácitos).

Impacto de los errores en la etapa de REQUISITOS

- ❑ El software resultante puede no satisfacer a los usuarios.
- ❑ Las interpretaciones múltiples de los requerimientos pueden causar desacuerdos entre clientes y desarrolladores.
- ❑ Puede gastarse tiempo y dinero construyendo el sistema que no cumple con la necesidad de los clientes.



*“La parte más difícil de construir un sistema es precisamente **saber qué construir**.*

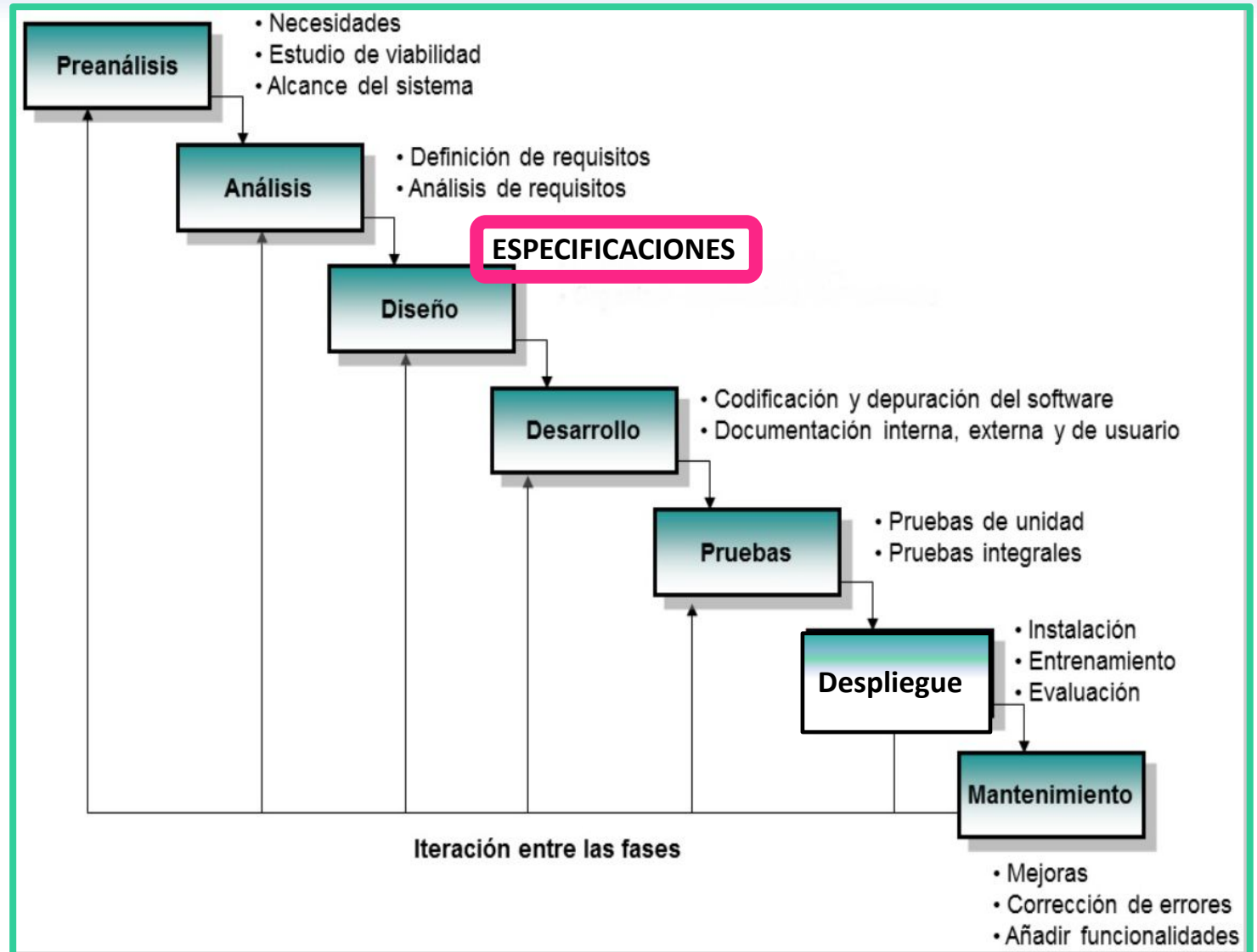
*Ninguna otra parte del trabajo conceptual es tan difícil como **DEFINIR** todos los requisitos detallados, incluyendo todas las interfaces con gente, máquinas y otros sistemas. Ninguna otra parte del trabajo afecta tanto el sistema si es hecha mal. Ninguna es tan difícil de corregir más adelante. Entonces, la tarea más importante que el ingeniero de software hace para el cliente es la extracción iterativa y el refinamiento de los requisitos del producto de software.”*

Frederick P. Brooks 1987

Etapa del ciclo de vida del software donde se definen los requisitos



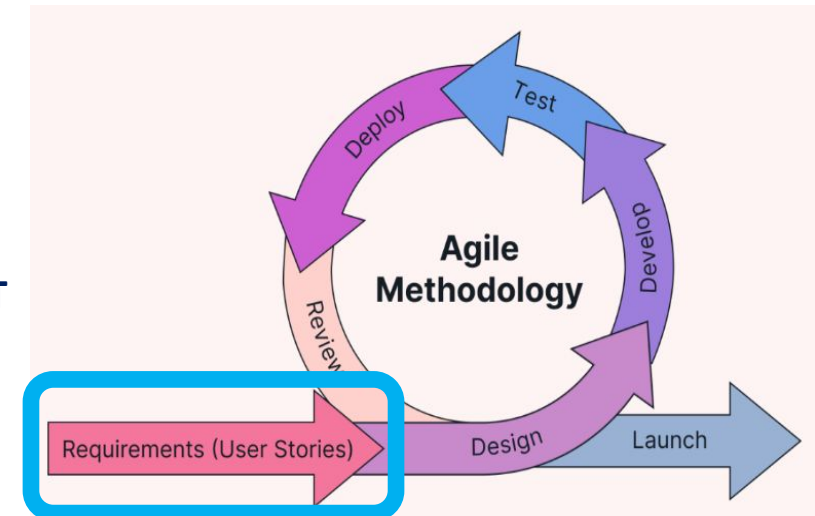
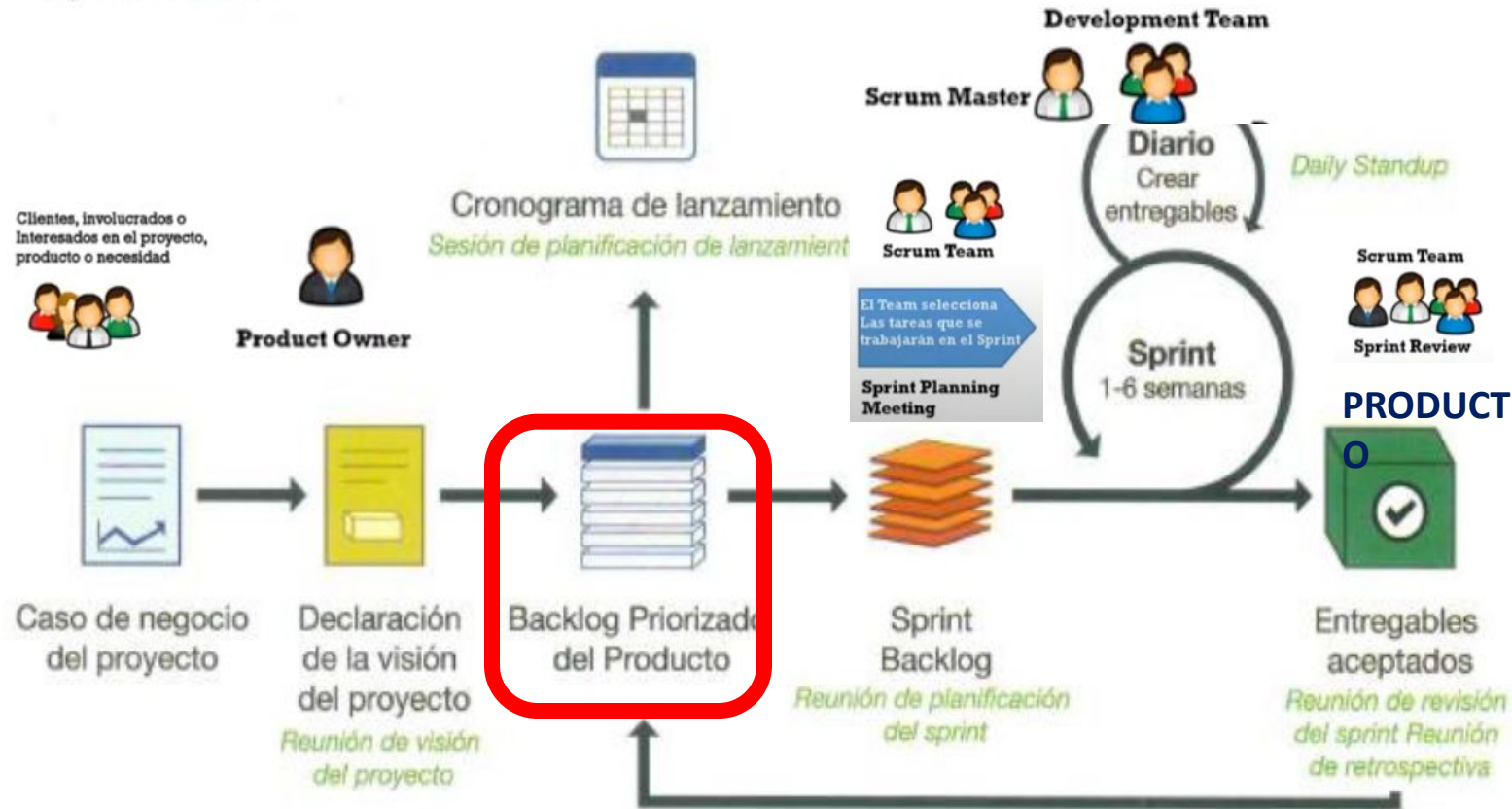
MODELO EN CASCADA



INGENIERÍA DE SOFTWARE

SCRUM FLUJO DEL PROCESO

Flujo de Scrum



DOMINIO DEL PROBLEMA

El dominio del problema **es el contexto, el entorno y el conjunto de actividades, reglas y necesidades donde se encuentra el problema** que queremos resolver mediante software.

Antes de pensar en cómo será el sistema (solución), **debemos entender qué problema existe, en qué área y cómo funciona ese mundo real.**

Es conocer la realidad antes de modelarla

La importancia de entender el dominio del problema

- Ayuda a **comprender el contexto real** antes de diseñar el software.
- Permite identificar las verdaderas **necesidades** del usuario (no solo lo que pide).
- Sirve como base para identificar stakeholders, procesos, restricciones externas y reglas del negocio.

¿Qué incluye el dominio del problema?

- ❑ **Procesos actuales:** cómo se hace hoy la actividad sin el nuevo sistema (manual, papel, sistemas antiguos).
- ❑ **Actores y roles:** personas o sistemas que interactúan con el proceso (internos y externos).
- ❑ **Objetivos de negocio:** qué quiere lograr la organización o usuario.
- ❑ **Políticas y Reglas del negocio:** Las políticas son declaraciones generales que guían la toma de decisiones y establecen el marco general de actuación de la empresa. Las reglas de negocio, por otro lado, son específicas y detalladas, definiendo cómo se deben llevar a cabo las acciones y operaciones dentro de la empresa
- ❑ **Restricciones externas:** leyes, regulaciones, compatibilidades técnicas, estándares.

DOMINIO DEL PROBLEMA

Ejemplo

Sistema a desarrollar: App de gestión de pedidos para una panadería.

Dominio del problema:

Proceso actual: toman pedidos por WhatsApp y anotan a mano.

Actores: vendedores, panaderos, clientes.

Objetivo: automatizar y evitar errores de pedidos, mejorar tiempos.

Políticas : gestión eficiente de recursos, la optimización de la cadena de suministro y la mejora de la experiencia del cliente.

Reglas: no se aceptan pedidos con menos de 24 hs de anticipación. Todo pedido debe ser despachado dentro de las 24 horas siguientes a su recepción.

Restricciones: deben poder imprimir tickets con una impresora ya existente.

EJERCITACIÓN EN CLASE

UTILIZAR EL DOCUMENTO DE ACTIVIDAD SUBIDO AL CANAL DE LA CLASE

BIBLIOGRAFÍA

- Robertson, Suzanne & James (2006). Dominando el proceso de requisitos.
- Wiegers, Karl & Beatty, Joy (2014). Requisitos de Software.
- Pohl, Klaus & Rupp, Chris (2015). Fundamentos de ingeniería de requisitos.
- Sommerville, I. (2016). Ingeniería de software (10.^a ed.). Pearson.
- Pressman, R. (2010) Ingeniería de software (7.^a ed.). Mc Graw Hill
- Vazquez, C. & Simoes, G. (2018). Ingenieria de Requisitos : Software Orientado al Negocio. ISBN: 9781729136683
- Edward Yourdon (1997). Análisis Estructurado Moderno



PREGUNTAS



DUDAS

COMENTARIOS